acri: A Client-Side Capability Resolver for Tool-Augmented Language Models

Piyush Sharma
INERATE
piyush@inerate.com github.com/INERATE/acri

***Abstract*—As the number of tools exposed to a language model grows past a small threshold, tool-selection accuracy degrades – Anthropic's own documentation places the cliff at 30–50 tools [1]. Anthropic and OpenAI ship server-side tool search to address this [1], [2]; providers without native tool search (Gemini, Ollama, vLLM, and any locally-hosted model) have no equivalent. We present acri, a client-side, provider-agnostic capability resolver: given a query and a registered tool corpus, it returns the k tools relevant to that call using BM25 lexical retrieval [3], with zero training and zero external dependencies. On a 50-query, 100-tool benchmark, acri improves tool-selection accuracy on every one of seven models tested across five vendors (Google, Anthropic, Meta, OpenAI, xAI) – live calls, gains of +2 to +10 points, one run per model. We further show, and this is the paper's central non-obvious result, that a naive "send fewer tokens" framing of this problem is often *wrong* once provider-side prompt caching is accounted for: cutting the tool-schema prefix at full price is more expensive than sending the full prefix at cache price unless the cut exceeds roughly tenfold. We report where our own benchmark breaks (recall@5 degrades from 100% to 92% at 500 tools) rather than hide it, and state explicitly what remains unmeasured before any version of this work could support a stronger claim. This is a technical report: it names its own evidence gaps rather than a set of results ready for peer review at a top venue.**

***Index Terms*—tool-augmented language models, retrieval-augmented generation, prompt caching, BM25, Model Context Protocol, tool selection**

I. Introduction

Tool-augmented language models select one function from a registered set on each turn. As that set grows, the model's task – reading N schemas and picking the right one – degrades in accuracy for a well-documented reason: schema tokens compete for the same attention budget as the rest of the prompt, and confusability between similarly-named or similarly-described tools rises with N. Anthropic's tool-use documentation names 30–50 tools as the point past which this degradation becomes visible in practice, and ships *tool search* – server-side retrieval over a large tool catalog – as the mitigation [1]. OpenAI ships an equivalent [2].

Gemini does not. Ollama does not. vLLM does not. A locally-hosted 8B model does not. Building against these gets no mitigation for free, and re-implementing server-side retrieval per provider is not a reasonable ask of an application author.

acri is that layer, implemented once, client-side: `pip install pyacri`, `import acri`, no gateway process, no training step, no provider lock-in. It sits between application code and whichever provider SDK is already in use, and decides – per call – which k of the registered tools the model actually sees.

A. What this paper claims, and what it does not

Early internal drafts of this project accumulated an unsupportable claim set: lower cost, faster execution, better memory, higher accuracy, no hallucination, all attributed to the same mechanism. Table I sorts every claim actually made in this work by whether it survives scrutiny, including the claims we explicitly do *not* make.

TABLE I
Claims, sorted by defensibility

Claim	Status
Improves tool-selection accuracy	Measured on seven models, five vendors: +2 to +10 points each (§IV-B).
Frees context capacity	Earnable, not separately measured.
Works where no native tool search exists	A fact, not an empirical claim.
Lowers API cost	False below $\sim$10$\times$ reduction against a cached baseline (§III-B).
Faster end-to-end execution	Not directly; resolver latency (p50 0.179 ms at 500 tools) is dwarfed by any network call.
Eliminates hallucinated tool calls	Not claimable; a smaller candidate set reduces likelihood, not possibility.

II. Related Work

Server-side tool search. Anthropic and OpenAI solve the identical problem for providers that ship it [1], [2]. acri targets the complement – every provider and self-hosted model that does not.

MCP's own filtering proposals. The Model Context Protocol's enhancement process has considered this problem directly. SEP-1300 ("Tool Filtering with Groups and Tags"), SEP-1821 ("Dynamic Tool Discovery"), and SEP-1881 ("Scope-Filtered Tool Discovery") were all closed as of late 2025 [4], [5], [6]. These are *static, server-declared* filters – an operator tags or groups tools ahead of time. acri performs *query-aware retrieval* – which tools are relevant changes per call, not per deployment. The mechanisms are complementary, not competing; the distinction matters for positioning, since the argument for query-aware retrieval narrows as server-declared filtering becomes more common.

Model routing. RouteLLM [7] addresses a related but distinct problem – selecting *which model* answers a query, not which tools it sees. It is the most-cited work in this space but has not been updated since August 2024. acri's own router component (§III-C) is scoped narrowly by comparison: route only stateless, prefix-free calls, once, before generation – informed by [8], which finds a lightweight pre-generation router outperforms optimal cascade policies on 4 of 5 evaluated datasets, because a cascade has already paid the cheap model's cost before deciding to escalate.

Adjacent implementations. `openclaw-toolsearch` [9] validates that this problem is recognized outside this work, but ships as a server-side proxy, not an importable client library, and has not taken the niche acri targets.

Serialization. TOON [10] claims 42.6% token reduction, measured against pretty-printed JSON. Against compact JSON – what every SDK actually transmits – the reduction is 14.5% on the same comparison, and TOON's own documentation states compact JSON wins for deeply-nested, non-uniform data, which describes JSON Schema tool definitions. We do not use TOON at the schema layer for this reason; acri's `press` component applies a TOON-style encoding at the *tool result* layer instead, where results are frequently large, uniform, and never cached.

III. System Design

A. Architecture

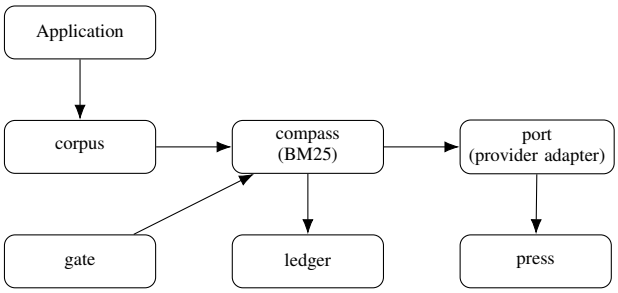

Fig. 1. acri's components. compass resolves once per task; gate is an advisory necessity check; press digests large tool results; ledger records every resolution for the evaluation in §IV.

acri decomposes into three components sufficient to form a complete, usable system on their own (Fig. 1): **corpus** (ingests tools from MCP servers, OpenAPI specs, or plain Python callables into one searchable index, built once and reused across a task), **compass** (the resolver: BM25 lexical retrieval [3], no training, no embedding model required), and **port** (thin per-provider adapters: Gemini, Vertex AI, OpenAI, Anthropic, and Bedrock natively, plus any OpenAI-compatible endpoint – Cloudflare Workers AI, OpenRouter, NVIDIA NIM, Grok, and self-hosted vLLM/Ollama/LM Studio – through one shared adapter, unchanged per endpoint). A **ledger** records every resolution decision (query, offered tools, model's selection, latency, corpus size) for the accuracy measurement in §IV, run across seven models, one run each (§V).

We use BM25 rather than dense embeddings deliberately: it needs no training data, no encoder model, and no vector index. Whether a dense retriever would outperform this on tool descriptions specifically is, as of this draft, an open, unmeasured question (§V); the claim we make is that lexical retrieval is *sufficient* to produce the accuracy result in §IV-B, not that it is optimal.

B. The central architectural argument: resolve once, never rewrite

This is the paper's primary non-obvious contribution, and the reason a naively-appealing design – re-run retrieval every conversational turn, swap the tool schema block each time – is wrong.

Every major LLM provider prices cached input tokens at roughly one-tenth of uncached input tokens, keyed on a stable prompt prefix. The tool-schema block sits at the front of the request. Rewriting it on every turn invalidates the provider's cache on every turn – the result is that a *smaller* prompt is billed at the *full* rate, while a larger, unchanged prompt is billed at the cached rate.

Formally, let a full, uncached tool block cost C tokens at price p, and a resolved block cost rC tokens ($r < 1$) at the same uncached price, versus the full block at the cached price $p/10$ (approximating typical provider cache discounts). Re-resolving every turn is cheaper only when:

$$rCp < C \cdot \frac{p}{10} \iff r < \frac{1}{10} \tag{1}$$

That is: cutting the tool block by less than roughly tenfold, while forfeiting the cache discount to do it, is a **net regression**, not a saving. The design Eq. (1) forces: **resolve once per task, write the tool block once, append after that – never rewrite a prefix that has already been sent.** The one sanctioned exception is `find_more_tools`, a capability always offered to the model so a bad initial resolution is recoverable mid-task via a normal appended tool call, rather than by rewriting history.

C. Supporting mechanisms

Four smaller components close specific gaps, each gated on evidence of need:

- **gate** – a necessity check (does this turn need a tool at all), implemented as a threshold on the BM25 score compass already computes, never a second trained classifier. Advisory only: it may influence what is offered, never veto what the model actually calls, since a wrongly-suppressed tool call is silent and unlogged while a spurious offer is cheap and logged.
- **press** – large tool *results* (not schemas) become a short digest plus a recoverable handle, addressing that JSON Schema tool definitions are TOON's documented worst case while tool results are frequently its best case.
- **sandbox** – CPU/memory/network/filesystem limits on stdio MCP servers via the host container engine. MCP

already provides process isolation; the addressed gap is *resource* limits, not process separation.

- **router** – scoped to stateless, prefix-free calls only, routed once before generation, never mid-stream. Mid-stream model switching was evaluated and rejected: stream-abort billing is unmeasurable at at least one major provider, and [11] measures a single clean handoff moving multi-turn instruction-following by double digits in *both* directions.

IV. Evaluation

All numbers in this section are produced by a script in `assay/`, this project's convention for where a reportable number is allowed to originate.

A. Recall@k

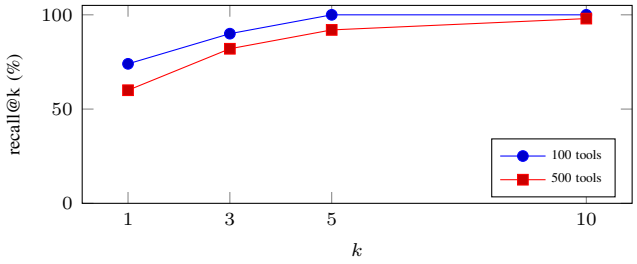

Fig. 2. Recall@k on a 100-tool corpus (52 gold queries) versus a 500-tool corpus, same queries unchanged.

Synthetic 100-tool corpus, 20 domains with deliberate cross-domain confusability, 50 hand-written queries phrased as a user would type them. Table II and Fig. 2 report recall@k both at 100 tools and after extending the corpus to 500 tools (41 new domains, same 52 gold queries, unchanged) – an honestly-reported degradation, not patched to look better. Query-by-query diagnosis attributes it to lexical competition: as the corpus grows, more tools share surface vocabulary with a given query, a limitation of BM25 specifically, not of the resolve-once architecture in §III-B. Latency at 500 tools: p50 0.179 ms, p95 0.285 ms – three orders of magnitude below any real provider network call.

TABLE II
Recall@k, 100 vs. 500 tools

k	recall@k (100)	recall@k (500)
1	74%	60%
3	90%	82%
5	100%	92%
10	100%	98%

B. Accuracy: does the model pick correctly

Recall measures whether the correct tool is *offered*; it says nothing about whether the model *chooses* it. `assay/accuracy.py` measures this directly: naive (all 100 tools shown) versus acri (top-5 shown), same 50 queries, live calls, no mocking – on seven models across five vendors (Google, Anthropic, Meta, OpenAI, xAI), so the direction of the effect is not an artifact of a single vendor's tuning, model generation, or hosting platform.

TABLE III
Tool-selection accuracy and per-call latency, seven models. Vertex: `claude-sonnet-4-6`, `claude-sonnet-5`. Azure: `grok-4-6`, `gpt-5-mini`.

Model	naive acc.	acri acc.	naive lat.	acri lat.
`gpt-5-mini`	70%	80%	7061 ms	5054 ms
`gemini-2.5-flash`	84%	92%	1792 ms	1596 ms
`claude-sonnet-4-6`	92%	96%	2668 ms	2873 ms
`claude-sonnet-5`	90%	92%	2745 ms	2366 ms
`grok-4-6`	96%	98%	4013 ms	4894 ms
`llama-3.1-70b-instruct`	96%	100%	2711 ms	1390 ms
`llama-3.3-70b-instruct`	98%	100%	2264 ms	1340 ms

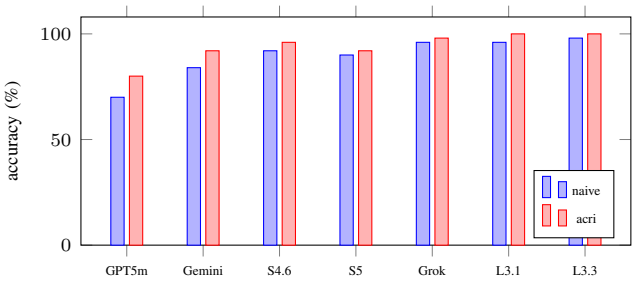

Fig. 3. Naive vs. acri accuracy, all seven models – every model improves under acri.

`gemini-2.5-flash` was corrected twice in public before reaching its current value; full history and the resulting ±2-point noise-floor estimate at n=50 are in §IV-D. The other six models are one live run each, discussed in §IV-C against that same noise floor.

C. Per-model detail, and the two results that break the pattern

`llama-3.1-70b-instruct` and `llama-3.3-70b-instruct` (both Cloudflare Workers AI): 4- and 2-point accuracy gaps, both smaller than §IV-B's ±2-point noise floor, both landing on a 100% ceiling at n=50. `claude-sonnet-5` (Vertex AI, `region=global`): 90%→92%, a 2-point gap sitting exactly at the noise floor – the weakest single result in this table. `grok-4-6` (Azure AI Foundry): 96%→98%; acri fixed both of naive's misses but introduced one new one on a query naive got right – a net win, not a strict superset, reported because a result that only ever helped the story would itself be the more suspicious one. `gpt-5-mini` (Azure AI Foundry): 70%→80%, the largest single gap and the lowest absolute accuracy of the seven – consistent with a smaller, cheaper model being more sensitive to schema bloat, not with acri behaving differently for it.

`claude-sonnet-4-6` and `grok-4-6` are the two of seven models whose acri-arm latency is *higher* than naive (2873 ms vs. 2668 ms; 4894 ms vs. 4013 ms) – breaking the "smaller prompt, faster call" pattern below. No explanation is confirmed for either; both are reported as-is.

One pattern worth naming, cautiously: the two largest accuracy gains (`gpt-5-mini` +10, `gemini-2.5-flash`

+8) belong to the two smallest/cheapest models tested, while every 70B-class or flagship model gains +2 to +4. Suggestive at n=7 models, not a controlled study of model size as a variable – a real pattern worth a dedicated benchmark, not yet a claim this paper makes.

Five of seven models show lower mean per-call latency on the acri arm; this is expected and is not the paper's architectural claim – a smaller prompt is processed faster on a single, cold, uncached call, independent of accuracy. It does not reflect the resolve-once-per-task caching §III-B is built around, a claim about repeated calls sharing a stable prefix, not any single call's raw speed.

D. Correction history, and what this evaluation does not show

A benchmark that always favors the tested hypothesis deserves the same suspicion as an unreceipted number, so `gemini-2.5-flash`'s full correction history is reported rather than only its final value. `resolver_miss=0` on the final run – every remaining acri-arm miss there is attributable to the model's choice, not to compass failing to offer the right tool. The naive arm's own score moved 2 points between runs 2 and 3 with no code change on its side; that movement is this benchmark's approximate noise floor at n=50, and every single-run result in §IV-C is judged against it.

TABLE IV
ACCURACY CORRECTION HISTORY (`GEMINI-2.5-FLASH`)

Run	naive	acri	Cause
1 (retracted)	24%	53%	All tool schemas had empty `properties`; model correctly declined to fabricate arguments. Not a real result.
2	72%	74%	Real JSON Schema on every tool; every query supplies each declared required value.
3 (current)	84%	92%	9 queries satisfied a tool's `required` fields on paper but withheld a value the action needed in practice; fixed by supplying it or promoting the field to required.

Seven models tested across five vendors – stronger than one model, but still one run per model: no computed confidence interval or paired significance test on any of the seven gaps, and all seven share the same synthetic corpus and 50-query set. No comparison against a dense-embedding retriever.

V. LIMITATIONS

Stated as gaps to close, not hedged around:

1) **Sample size.** 50–52 gold queries is small for a headline claim; no bootstrap confidence interval or paired significance test has been computed, on any of the seven models. Highest-priority gap.
2) **Model coverage.** Seven models evaluated across five vendors (Google, Anthropic, Meta, OpenAI, xAI), one run each. Breadth of vendors is not breadth of runs: no result has a computed confidence interval, one (`claude-sonnet-5`) sits at this benchmark's own noise floor, and two show a latency inversion (§IV-C).
3) **No embeddings baseline.** BM25 versus dense retrieval on tool descriptions is an open question this work has not resolved.
4) **Synthetic corpus.** Neither corpus was sourced from a real, in-production MCP catalog.
5) **Re-resolution triggers are unspecified.** §III-B forbids per-turn re-resolution but does not define what should trigger `find_more_tools`, or whether that is detectable without an extra model call.

VI. CONCLUSION

We present a client-side capability resolver motivated by a documented, provider-acknowledged degradation (tool-selection accuracy past 30–50 tools) and targeted at the class of providers that ship no server-side mitigation. The retrieval mechanism itself is a standard BM25 index; the contribution is the observation that provider-side prompt caching inverts the naive cost argument for doing this at all (Eq. 1). We report measured tool-selection accuracy improvements on seven models across five vendors (Table III; gains of +2 to +10 points, one run each), an honestly-reported recall degradation at 5× corpus scale (100%→92% at k=5), and five specific, unresolved gaps – sample size and model coverage foremost – that separate this technical report from a claim ready for peer review at a top venue.

REPRODUCTION

`pip install -e ".[dev]"; python -m assay.recall; python -m assay.scale; python -m assay.accuracy --provider gemini` (needs `GEMINI_API_KEY`); or `--provider cloudflare --model @cf/meta/llama-3.1-70b-instruct` or `llama-3.3-70b-instruct` (needs `CLOUDFLARE_ACCOUNT_ID`, `CLOUDFLARE_API_TOKEN`); or `--provider vertex-claude --model claude-sonnet-5` or `claude-sonnet-4-6` (needs `GOOGLE_CLOUD_PROJECT`, `GOOGLE_APPLICATION_CREDENTIALS`); or `--provider azure-grok --model grok-4-6` or `--provider azure-openai --model gpt-5-mini` (needs `AZURE_AI_ENDPOINT`, `AZURE_AI_API_KEY`); `python -m assay.diagnose`. Source: `github.com/INERATE/acri`. Package: `pip install pyacri`. An animated walkthrough of the mechanism in §III-B is at `inerate.github.io/acri/demo`.

REFERENCES

[1] Anthropic, "Tool search tool," https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-search-tool, 2026, states a 30–50 tool accuracy-degradation threshold. Accessed 2026-08-27.

[2] OpenAI, "Using tools," https://developers.openai.com/api/docs/guides/tools, 2026, documents `defer_loading` and `tool_search`. Accessed 2026-08-27.

[3] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: Bm25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[4] Model Context Protocol, "Sep-1300: Tool filtering with groups and tags," https://github.com/modelcontextprotocol/modelcontextprotocol, 2025, closed 2025-12-01, `state_reason: completed`.
[5] ——, "Sep-1821: Dynamic tool discovery," https://github.com/modelcontextprotocol/modelcontextprotocol, 2025, closed, `state_reason: completed`.
[6] ——, "Sep-1881: Scope-filtered tool discovery," https://github.com/modelcontextprotocol/modelcontextprotocol, 2025, closed, `state_reason: completed`.
[7] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "Routellm: Learning to route llms with preference data," *arXiv preprint arXiv:2406.18665*, 2024.
[8] Nguyen and Diao, "Is escalation worth it? a decision-theoretic characterization of llm cascades," arXiv:2605.06350, 2026.
[9] PyPI, "openclaw-toolsearch," https://pypi.org/project/openclaw-toolsearch/, 2026, first release 2026-03-07; a server-side proxy, not a client library.
[10] J. Schopplich, "Toon: Token-oriented object notation," https://github.com/toon-format/toon, 2026, claims 42.6% fewer tokens vs. pretty JSON; compact JSON wins on nested data.
[11] Kumar and Reyes, "Evaluating performance drift from model switching in multi-turn llm systems," arXiv:2603.03111, 2026.